

1. What This Project Actually Does

In plain words: you drop a photo or video into a folder called `inbox`. You run one command. The program looks at that photo, asks Google's Gemini AI to write a LinkedIn caption about it (in your voice, with your hashtags), shows you the caption in the terminal so you can approve or regenerate it, and — once you say "yes" — uploads it straight to your LinkedIn profile as a real post. Afterwards it moves the file into a "posted" folder so it's never posted twice.

- 1. **Find media** — look inside the `inbox/` folder for a photo or video.
- 2. **Ask for a hint** — you type a short note about what today's post should mention (optional).
- 3. **Generate caption** — send the photo + hint to Gemini AI, get back a written caption.
- 4. **You approve it** — terminal shows the caption; you type `y / r / h / q`.
- 5. **Post to LinkedIn** — upload the media, then publish the post using LinkedIn's API.
- 6. **Archive the file** — move the photo/video into `posted/<today's date>/` so it won't repost.

There are two separate Python files in this project, and they are run at completely different times:

File	When you run it	What it's for
<code>linkedin_auth_helper.py</code>	Only once (or every ~60 days)	Logs into LinkedIn through your browser and gets a secret "access token" that lets the other script post on your behalf.
<code>daily_poster.py</code>	Every day (this is the "agent")	Does the actual work: reads a photo, writes a caption with AI, and publishes the LinkedIn post.

Why two files? Because logging in (authorization) and posting are different jobs. You don't want to open a browser and log in every single day — so you log in once, save the resulting "token" (like a temporary key) into a file called `.env`, and every day's script just reuses that saved key.

2. How the Two Files Relate to Each Other

Neither file imports the other — they don't call each other's functions directly. Instead, they are connected through a shared file called `.env`, which stores secret values as plain text (like a settings file). Think of `.env` as a locked notebook that both scripts are allowed to read from.

- **Step A (run once):** You run `linkedin_auth_helper.py`. It opens LinkedIn in your browser, you click "Allow", and it prints two values: `LINKEDIN_ACCESS_TOKEN` and `LINKEDIN_PERSON_URN`.
- **Step B:** You paste those two values into `.env` by hand.
- **Step C (run daily):** You run `daily_poster.py`. At the very top, the line `load_dotenv()` reads `.env` into memory, so `os.environ["LINKEDIN_ACCESS_TOKEN"]` now works inside the script.

So the connection isn't "Python file calls Python file" — it's "Python file writes secrets to a text file, another Python file reads them back later." This is a very common pattern for keeping passwords/keys out of your actual code.

Security note: `.env` contains real secret keys (Gemini API key, LinkedIn tokens, client secret). This file should never be uploaded publicly (e.g. to GitHub) without protection — normally you'd add it to a `.gitignore` file, or if using GitHub Actions, store these values as encrypted "Secrets" in the repository settings instead of committing the actual `.env` file.

3. Every Import Explained (in Plain English)

At the top of every Python file, `import` lines pull in extra tools other people already built, so you don't have to write everything from scratch. Here is every import used across both files:

3.1daily_poster.py imports

Import	Built-in or external?	What it's used for here
<code>os</code>	Built into Python	Reading environment variables, e.g. <code>os.environ["GEMINI_API_KEY"]</code> — this is how the script reads secrets from <code>.env</code> .
<code>sys</code>	Built into Python	Used once, to force the terminal to correctly display special characters/emojis (<code>sys.stdout.reconfigure</code>).
<code>shutil</code>	Built into Python	"Shell utilities" — used to check if a program (ffmpeg) exists, and to move files into the archive folder.
<code>subprocess</code>	Built into Python	Lets Python run another program from the command line (here, it runs <code>ffmpeg</code> to grab a frame from a video).
<code>mimetypes</code>	Built into Python	Figures out the file type (e.g. "image/jpeg") from a filename, needed when sending the photo to the AI.
<code>datetime.date</code>	Built into Python	Gets today's date, used for log entries and for naming the "posted" archive folder.
<code>pathlib.Path</code>	Built into Python	A modern, easy way to work with file/folder paths (instead of joining strings manually).
<code>requests</code>	External (installed via pip)	Sends HTTP requests to LinkedIn's servers — this is literally how the post gets uploaded to the internet.
<code>google.generativeai</code>	External (Google's Gemini SDK)	The official library to talk to Google's Gemini AI model and generate the caption text.
<code>google.generativeai.types</code>	External (part of the same SDK)	Provides helper structures like <code>Part</code> and <code>GenerateContentConfig</code> needed to build the AI request correctly.
<code>dotenv.load_dotenv</code>	External (python-dotenv)	Reads the <code>.env</code> file and loads its values so <code>os.environ[...]</code> can see them.

3.2linkedin_auth_helper.py imports

Import	Built-in or external?	What it's used for here
<code>os</code>	Built into Python	Reads <code>LINKEDIN_CLIENT_ID</code> and <code>LINKEDIN_CLIENT_SECRET</code> from the environment.
<code>http.server</code>	Built into Python	Spins up a tiny temporary local web server on your own computer, so LinkedIn has somewhere to send you back to after you log in.
<code>urllib.parse</code>	Built into Python	Builds and reads URL query strings (e.g. turning a dictionary of parameters into a proper web link).
<code>webbrowser</code>	Built into Python	Automatically opens your default browser to LinkedIn's login/authorize page.
<code>requests</code>	External	Exchanges the temporary "authorization code" for a real access token, and fetches your LinkedIn user ID.

`dotenv.load_dotenv`

External

`pip`Loads your Client ID/Secret from `.env` so this script can use them.

Rule of thumb: if you can use it without installing anything (like `os` , `sys` , `datetime`), it's "built-in." If it needed `pip install` at some point, it's "external" — someone else wrote that code and shared it publicly.

4. daily_poster.py — Line by Line

This is the file that runs every day. We'll go top to bottom, function by function.

4.1 Setup section (lines 1–29)

```
Line 1-12  import os, sys, shutil, subprocess, mimetypes, date, Path, requests,
           genai, types, load_dotenv
Line 14    load_dotenv() -> reads .env into memory
Line 16-19 try/except around sys.stdout.reconfigure()
           -> makes sure emojis/special characters print correctly in the
           terminal, without crashing on older systems
Line 21    BASE_DIR = Path(__file__).parent
           -> finds the folder this .py file lives in, so paths work no
           matter where you run the script from
Line 22-24 INBOX_DIR, POSTED_DIR, LOG_FILE
           -> define three important paths: where new photos go in,
           where posted photos are archived, and where logs are written
Line 26-27 IMAGE_EXTS, VIDEO_EXTS
           -> two "sets" (a list with no duplicates) of allowed file
           extensions, so the script knows what counts as a photo/video
Line 29    GEMINI_MODEL = "gemini-3.1-flash-lite"
           -> the exact AI model name that will write the captions
```

Beginner note: anything written as `CONSTANT_NAME = value` at the top of a file (all capital letters) is just a fixed setting used later in the code. It's not a function — it's a labelled box holding a value.

`def log(msg: str):` lines 32-36

What it does: prints a message to the terminal AND saves it (appends) to `poster.log`, with today's date stuck on the front.

- **Line 33:** builds the text `[2026-07-12] your message here`.
- **Line 34:** `print(line)` — shows it in the terminal right now.
- **Line 35-36:** opens `poster.log` in "append" mode (`"a"`, meaning add to the end without erasing what's already there) and writes the line, followed by a newline.

Why it matters: every other function calls `log(...)` whenever something important happens, so you get a permanent history file of everything the agent has ever done — useful when it runs automatically overnight and you're not watching.

`def get_todays_file():` lines 39-44

What it does: looks inside the `inbox` folder and returns the first photo/video it finds.

- **Line 40:** `INBOX_DIR.mkdir(exist_ok=True)` — creates the `inbox` folder if it doesn't already exist (`exist_ok=True` just means "don't crash if it's already there").
- **Line 41:** `sorted(INBOX_DIR.iterdir())` — lists every file inside `inbox`, alphabetically.
- **Line 42:** checks if the file's extension (like `.png`) is in the allowed image or video list. `IMAGE_EXTS | VIDEO_EXTS` combines both sets into one using the `|` (OR) operator.
- **Line 43:** as soon as one valid file is found, it's returned immediately — the function stops there.
- **Line 44:** if the loop finishes with nothing found, it returns `None` (Python's way of saying "nothing here").

`def get_hint() -> str:` lines 47-53

What it does: asks you, in the terminal, to type a short note about today's post (e.g. "leg day PR").

- **Line 51:** `input("> ")` pauses the program and waits for you to type something and press Enter.
- **Line 52-53:** if the input can't be read for some reason (like running with no terminal attached), it just returns an empty string instead of crashing.

The `-> str` in the function definition is a "type hint" — it just tells other programmers (and you) that this function always gives back text (a string). It doesn't change how the code runs.

`def extract_video_frame(video_path: Path) -> Path | None:` lines 56-70

What it does: if today's file is a video, this grabs a single still image from it (1 second in), because the AI can only "look at" images, not videos.

- **Line 57:** `shutil.which("ffmpeg")` checks whether the free tool `ffmpeg` is installed on the computer. If not, the function gives up early and returns `None`.
- **Line 59:** builds a new filename like `myvideo.preview.jpg` next to the original video.
- **Line 61-67:** runs `ffmpeg` as an external command (via `subprocess.run`) with instructions: open the video, jump to the 1-second mark (`-ss 00:00:01.000`), grab exactly 1 frame (`-vframes 1`), and save it as a jpg.
- **Line 68:** if that image file now exists on disk, return its path; otherwise return `None`.
- **Line 69-70:** if anything goes wrong (ffmpeg missing arguments, corrupted video, etc.) it's caught by `except Exception` and treated the same as "couldn't do it" — quietly returns `None` rather than crashing the whole program.

`PROFILE_CONTEXT (a big text block)` lines 74-88

This isn't a function — it's a long piece of text (a "string") written once and reused. It describes who Rudra is, his skills, and the tone his posts should sound like. It gets pasted into every AI prompt so Gemini always "remembers" the writing style, even though the AI itself has no real memory between runs.

```
def generate_caption(media_file: Path, hint: str) -> str: lines 93-201
```

What it does: this is the heart of the AI part. It sends the photo (plus instructions and your hint) to Gemini, and gets back a finished caption as text.

- **Line 95:** `genai.Client(api_key=...)` — opens a connection to Google's Gemini service using your secret key from `.env`.
- **Line 97-103:** if the file is a video, it calls `extract_video_frame` (the function above) to get a still image, since the AI needs an image to "see" the post.
- **Line 105-127:** builds the actual instructions (the "prompt") sent to the AI — a detailed set of rules like "start with a playful intro line," "write in first person," "end with 6-8 hashtags," etc. `PROFILE_CONTEXT` is glued in at the top.
- **Line 129-140:** if you typed a hint earlier, it's appended onto the prompt so the AI knows what today's post is actually about.
- **Line 142-155:** builds a list called `contents` that Gemini needs. If there's a valid image, its raw bytes are added first (`types.Part.from_bytes`) with the correct mime type (e.g. "image/jpeg"), then the text prompt is added after it.
- **Line 157-164:** `client.models.generate_content(...)` actually sends everything to Google's servers and waits for the AI's reply. `max_output_tokens=2048` limits how long the reply can be, and `thinking_budget=0` tells the model to answer directly without extra "internal reasoning" steps (faster and cheaper).
- **Line 166-171:** checks the AI's `finish_reason`. If the response got cut off mid-way instead of ending naturally, this raises an error on purpose so the calling code knows to retry.
- **Line 173-176:** pulls out the actual text of the reply and removes extra blank space. If it's empty, raises an error.
- **Line 178-181:** sometimes AI models wrap answers in markdown code fences (triple backticks). This strips those out if present.
- **Line 183-198:** a safety check — a list of `bad_phrases` that would suggest the AI leaked its own internal instructions instead of writing a real caption (e.g. the words "JSON" or "Final Polish" showing up). If any of these appear, the function raises an error rather than posting garbage.
- **Line 201:** returns the finished, cleaned-up caption text.

```
def _linkedin_headers(token: str) -> dict: lines 203-209
```

What it does: builds the small "header" information that must be attached to every request sent to LinkedIn's servers — like an ID card presented alongside each request. Includes the secret access token, which LinkedIn API version to use, and that the data being sent is in JSON format.

The underscore at the start of the name (`_linkedin_headers`) is a Python convention meaning "this is a helper function only meant to be used inside this file."

```
def post_to_linkedin(media_file: Path, caption: str):
```

lines 212-251

What it does: uploads the photo/video to LinkedIn, then publishes the actual post with the caption. This talks to LinkedIn in two stages, which is how their API works.

- **Line 213-214:** reads your LinkedIn access token and your account ID (person URN) from `.env`.
- **Line 215-216:** decides whether this is a photo or video upload, since LinkedIn has separate endpoints (web addresses) for each.
- **Line 218-223:** Stage 1 — calls LinkedIn's "initializeUpload" endpoint. This tells LinkedIn "I'm about to upload a file," and LinkedIn replies with a special one-time upload web address (`uploadUrl`) plus an ID for the media (`asset_urn`).
- **Line 224-226:** pulls those two values out of LinkedIn's JSON reply.
- **Line 228-230:** Stage 2 — opens the actual photo/video file in "read bytes" mode (`"rb"`) and uploads its raw content to that one-time `uploadUrl` using an HTTP `PUT` request.
- **Line 232-244:** builds the actual post content as a dictionary: who's posting, the caption text, that it should be public, that it goes to the main feed, and which uploaded media ID to attach.
- **Line 245-249:** sends this post data to LinkedIn's `/rest/posts` endpoint — this is the moment the post actually goes live.
- **Line 250:** `raise_for_status()` appears after every request — if LinkedIn responds with an error code, this immediately stops the function and raises an exception rather than silently continuing.
- **Line 251:** logs success.

```
def archive_file(media_file: Path):
```

lines 254-257

What it does: moves the just-posted file out of `inbox/` into `posted/<today's date>/`, so the same photo never gets picked up and posted again tomorrow.

- **Line 255:** builds a folder name using today's date, e.g. `posted/2026-07-12/`.
- **Line 256:** creates that folder (and any missing parent folders, thanks to `parents=True`) if it doesn't exist yet.
- **Line 257:** `shutil.move(...)` physically moves the file there.

```
def generate_caption_with_retries(media_file, hint) -> str:
```

lines 260-270

What it does: a safety wrapper around `generate_caption`. AI services sometimes fail temporarily (server overload, a cut-off response). Instead of giving up on the first failure, this tries up to 3 times before finally giving up.

- **Line 264:** a loop that runs 3 times (`range(3)` gives 0, 1, 2).
- **Line 265-266:** tries to generate the caption; if it works, returns immediately.
- **Line 267-269:** if it fails, remembers the error and logs which attempt number failed, then loops around to try again.
- **Line 270:** if all 3 attempts fail, re-raises the last error so the caller knows it genuinely couldn't be done.

`def main():` `lines 273-318`

What it does: this is the "conductor" function — it doesn't do the detailed work itself, it just calls all the other functions in the right order. This is the function that actually runs when you launch the script.

- **Line 274-277:** gets today's file; if there isn't one, logs that and stops immediately (`return` exits the function early).
- **Line 279:** asks you for a hint via the terminal.
- **Line 281-311:** a `while True:` loop — this repeats forever until something inside it explicitly breaks out. This is what powers the "approve or regenerate" feature:
 - Generates a caption (with retries).
 - Prints it nicely between rows of `=` characters so it stands out in the terminal.
 - Asks you to type `y`, `r`, `h`, or `q`.
 - `y` (yes) → `break` exits the loop, moving on to actually post it.
 - `r` (regenerate) → `continue` jumps back to the top of the loop and tries again with the same hint.
 - `h` (hint) → asks for a brand-new hint, then loops back to regenerate with it.
 - `q` (quit) → logs that you cancelled, and exits the whole function (the file stays safely in `inbox/` for tomorrow).
 - Anything else typed → prints "Didn't catch that" and loops back to ask again.
- **Line 313-318:** once you've approved a caption, tries to actually post it and archive the file. If posting fails for any reason (e.g. LinkedIn's servers are down, token expired), it logs the error and deliberately leaves the file in `inbox/` so tomorrow's run will simply try again with it.

`if __name__ == "__main__": main()` `lines 321-322`

This is a very common Python pattern. It means: "only run the `main()` function if this file was run directly (e.g. `python daily_poster.py`) — not if some other file imports it." It's the literal starting point of the whole program.

5. linkedin_auth_helper.py — Line by Line

This file is run rarely (once, then again every ~60 days when the token expires). Its only job is to get permission from LinkedIn and hand back two secret values.

5.1 Setup (lines 1-28)

```
Line 1-14  A big comment (triple-quoted string) explaining setup steps:
           create a LinkedIn developer app, add "Share on LinkedIn" product,
           set the redirect URL, and copy Client ID/Secret into .env
Line 16-23 Imports + load_dotenv() (same idea as the other file)
Line 25-26 CLIENT_ID / CLIENT_SECRET read from .env
Line 27    REDIRECT_URI = "http://localhost:8765/callback"
           -> after logging in, LinkedIn will send the browser back to
           this address on YOUR OWN computer
Line 28    SCOPES = "openid profile w_member_social"
           -> the specific permissions being requested: basic identity info
           and the ability to post on your behalf
Line 30    auth_code = {}
           -> an empty dictionary that will temporarily store the login
           code once LinkedIn sends it back
```

`class CallbackHandler(http.server.BaseHTTPRequestHandler):` Lines 33-48

What it does: defines how a tiny temporary web server (running on your own computer) should respond when LinkedIn sends the browser back to it after you approve the login.

- **Line 34-45 (`do_GET`):** runs automatically whenever a browser visits this local server. It reads the URL LinkedIn redirected to, pulls out the `code` parameter from the web address (`urllib.parse` is what decodes this), stores it in the `auth_code` dictionary, and shows a friendly "Success!" message in the browser tab. If no code was found, it shows an error instead.
- **Line 47-48 (`log_message`):** overrides the server's default noisy logging so it stays silent in the terminal.

Beginner note: a `class` is a blueprint for creating an object that bundles related behavior together. Here, it's just "the rulebook for how this mini local server behaves when it receives a visit."

`def main():` Lines 51-103

- **Line 52-62:** builds the LinkedIn login web address by combining the base authorize URL with parameters (client ID, redirect address, requested scopes) using `urllib.parse.urlencode`.
- **Line 63-65:** prints the link and uses `webbrowser.open(...)` to automatically open it in your default browser.
- **Line 67:** starts the tiny local server on port 8765 (matches the redirect address).
- **Line 68:** `server.handle_request()` pauses the whole script here and waits — literally does nothing else — until exactly one web request comes in (which will be LinkedIn redirecting your browser back after you click "Allow").
- **Line 70-72:** if somehow no code was received, prints an error and stops.
- **Line 75-86:** exchanges the temporary authorization code for a real, longer-lived `access_token` by sending a POST request to LinkedIn's token endpoint, including your Client ID and Secret to prove this request is legitimate.
- **Line 89-96:** uses that access token to ask LinkedIn "who am I?" via the `/v2/userinfo` endpoint, then builds your account's unique ID string, formatted as `urn:li:person:XXXXX`.
- **Line 98-102:** prints both final values clearly so you can copy-paste them into `.env`, plus a reminder that the token expires in 60 days.

```
if __name__ == "__main__": main()
```

lines 105-106

Same pattern as before — this is what actually kicks things off when you run `python linkedin_auth_helper.py` directly.

6. The Full Journey of One Run, Start to Finish

Here's exactly what happens, in order, when you run `python daily_poster.py` on a day when there's a photo waiting in `inbox/`:

1. Python loads the file top to bottom, defining all the functions (nothing runs yet, just gets prepared).
2. `load_dotenv()` reads your secret keys from `.env` into the program's memory.
3. `if __name__ == "__main__":` is true (you ran this file directly), so `main()` is called.
4. `main()` calls `get_todays_file()` → finds your photo in `inbox/`.
5. `main()` calls `get_hint()` → you type a short note, or press Enter to skip.
6. `main()` calls `generate_caption_with_retries()`, which calls `generate_caption()`, which contacts Gemini AI and gets back a caption.
7. The caption is printed to the terminal for you to read.
8. You type `y` to approve (or `r` / `h` to try again, or `q` to cancel).
9. `main()` calls `post_to_linkedin()`, which uploads the image and then publishes the post using LinkedIn's API.
10. `main()` calls `archive_file()`, moving the photo into `posted/2026-07-12/`.
11. Every important moment along the way was also written into `poster.log` by the `log()` function.

Automation: according to your project's setup, this whole run is triggered automatically every day on a schedule via GitHub Actions (a free tool from GitHub that can run scripts on a timer in the cloud), so you don't have to manually type the command yourself each day — although the terminal approval step (typing y/r/h/q) still needs a live terminal, which is why this project is described as having an "interactive terminal approval loop."

7. Files and Folders This Project Creates on Its Own

Path	Created by	Purpose
<code>inbox/</code>	<code>get_todays_file()</code> , line 40	Where you manually drop today's photo/video before running the script.
<code>posted/<date>/</code>	<code>archive_file()</code> , line 256	Where successfully-posted media gets moved, organized by the date it was posted.
<code>poster.log</code>	<code>log()</code> , line 35	A running text history of everything the script has ever logged — errors, successes, cancellations.
<code>.env</code>	You create this manually	Stores all secret keys: Gemini API key, LinkedIn access token, person URN, client ID/secret.

8. Python Concepts Used, Explained Simply

Concept	Plain-English meaning
<code>def function_name():</code>	Defines a reusable block of instructions you can "call" (run) by name whenever you need it.
<code>return value</code>	Hands a result back to whoever called the function, and immediately stops running the rest of that function.
<code>try / except</code>	"Attempt this code — if it fails/crashes, don't stop the whole program, run this backup plan instead."
<code>with open(file) as f:</code>	Safely opens a file, and automatically closes it again once you're done — even if something goes wrong.
<code>for item in list:</code>	Repeats a block of code once for every item in a collection (like every file in a folder).
<code>while True: ... break</code>	Repeats forever until you explicitly tell it to stop with <code>break</code> — used here for the "keep regenerating until approved" loop.
<code>dict</code> (dictionary, e.g. <code>{"key": "value"}</code>)	A collection of labeled values you look up by name, like a real dictionary — used constantly to build data sent to LinkedIn/Gemini.
<code>os.environ["NAME"]</code>	Reads a secret/setting by name from the environment (ultimately loaded from <code>.env</code>).
Type hints, e.g. <code>-> str</code> or <code>path: Path</code>	Notes (not enforced by Python itself) telling you what type of data goes in/out of a function — purely for readability.
<code>raise Exception("message")</code>	Deliberately stops the current function and reports an error, which can then be caught by a <code>try/except</code> elsewhere.
HTTP requests (<code>requests.get/post/put</code>)	The way any program talks to a website/server over the internet — asking for data, or sending data to be saved.

9. Cheat Sheet — One-Line Summary of Every Function

Function	File	One-line job
<code>log()</code>	daily_poster.py	Prints and saves a timestamped message.
<code>get_todays_file()</code>	daily_poster.py	Finds the first photo/video sitting in <code>inbox/</code> .
<code>get_hint()</code>	daily_poster.py	Asks you to type a quick note about today's post.
<code>extract_video_frame()</code>	daily_poster.py	Grabs one still image from a video using ffmpeg.
<code>generate_caption()</code>	daily_poster.py	Sends the photo + hint to Gemini AI and returns a written caption.
<code>_linkedin_headers()</code>	daily_poster.py	Builds the ID-card-like headers needed for every LinkedIn API call.
<code>post_to_linkedin()</code>	daily_poster.py	Uploads the media, then publishes the actual LinkedIn post.
<code>archive_file()</code>	daily_poster.py	Moves the posted file into <code>posted/<date>/</code> .
<code>generate_caption_with_retries()</code>	daily_poster.py	Tries <code>generate_caption()</code> up to 3 times before giving up.
<code>main()</code>	daily_poster.py	Runs the whole daily flow, start to finish.
<code>CallbackHandler.do_GET()</code>	linkedin_auth_helper.py	Catches LinkedIn's login redirect and grabs the authorization code.
<code>main()</code>	linkedin_auth_helper.py	Opens the LinkedIn login page, waits for approval, and exchanges it for a real access token + your account ID.

Generated as a personal, plain-English project walkthrough. Every line reference matches the code exactly as uploaded, so you can follow along in your editor while reading this.